

Introduction to Databases

(INFR10080)

(Relational Algebra)

Instructor: Yang Cao

(Fall 2025)



THE UNIVERSITY
of EDINBURGH

Changelog

v25.0 Initial version

Data model

- A **relation** is a **set** of **records**
over the same **set** of (distinct) attribute names
- A **record** is a **total function** from attribute names to values

Schema

- Set of **relation names**
- Set of **distinct attributes** for each table
Note that columns are not ordered

Instance

- Actual data (that is, the records in each relation)

2/21

Relational algebra

Procedural query language

A relational algebra expression

- takes as input one or more relations
- applies a **sequence of operations**
- returns a relation as output

Operations:

Projection (π)

Selection (σ)

Product ($\times$)

Renaming (ρ)

Union ($\cup$)

Intersection ($\cap$)

Difference ($-$)

The application of each operation results in a new (virtual) relation that can be used as input to other operations

3/21

Projection

- **Vertical operation**: choose some of the columns
- **Syntax**: $\pi_{\text{set of attributes}}(\text{relation})$
- $\pi_{A_1, \dots, A_n}(R)$ takes only the values of attributes $A_1, \dots, A_n$ for each tuple in R

Customer

CustID	Name	City	Address
cust1	Renton	Edinburgh	2 Wellington Pl
cust2	Watson	London	221B Baker St
cust3	Holmes	London	221B Baker St

$\pi_{\text{Name, City}}(\text{Customer})$

Name	City
Renton	Edinburgh
Watson	London
Holmes	London

4/21

Selection

- **Horizontal operation**: choose rows satisfying some condition
- **Syntax**: $\sigma_{\text{condition}}(\text{relation})$
- $\sigma_{\theta}(R)$ takes only the tuples in R for which θ is satisfied

term := attribute | constant

θ := **term** **op** **term** with **op** $\in \{=, \neq, >, <, \geq, \leq\}$
| $\theta \wedge \theta$ | $\theta \vee \theta$ | $\neg \theta$

5/21

Example of selection

Customer

CustID	Name	City	Age
cust1	Renton	Edinburgh	24
cust2	Watson	London	32
cust3	Holmes	London	35

$\sigma_{\text{City} \neq \text{'Edinburgh'} \wedge \text{Age} < 33}(\text{Customer})$

CustID	Name	City	Age
cust2	Watson	London	32

6/21

Efficiency (1)

Consecutive selections can be combined into a single one:

$$\sigma_{\theta_1}(\sigma_{\theta_2}(R)) \equiv \sigma_{\theta_1 \wedge \theta_2}(R)$$

↳ is equivalent to: two queries Q_1 and Q_2 are **equivalent** if Q_1 returns the same answers as Q_2 on every database

Example

$$Q_1 = \sigma_{\text{City} \neq \text{'Edinburgh'}}(\sigma_{\text{Age} < 33}(\text{Customer}))$$

$$Q_2 = \sigma_{\text{City} \neq \text{'Edinburgh'} \wedge \text{Age} < 33}(\text{Customer})$$

$Q_1 \equiv Q_2$ but Q_2 **faster** than Q_1 in general

7/21

Efficiency (2)

Projection can be pulled in front of selection

$$\sigma_{\theta}(\pi_{\alpha}(R)) \equiv \pi_{\alpha}(\sigma_{\theta}(R))$$

only if all attributes mentioned in θ appear in α

Example

$$Q_1 = \pi_{\text{Name, City, Age}}(\sigma_{\text{City} \neq \text{'Edinburgh'} \wedge \text{Age} < 33}(\text{Customer}))$$

$$Q_2 = \sigma_{\text{City} \neq \text{'Edinburgh'} \wedge \text{Age} < 33}(\pi_{\text{Name, City, Age}}(\text{Customer}))$$

Question: Which one is more efficient?

8/21

Cartesian product

$R \times S$ **concatenates** each tuple of R with all the tuples of S

Note: the relations must have **disjoint** sets of attributes

Example

R	A	B	$\times$	S	C	D	=	$R \times S$	A	B	C	D
	1	2			1	a			1	2	1	a
	3	4			2	b			1	2	2	b
					3	c			1	2	3	c
									3	4	1	a
									3	4	2	b
									3	4	3	c

Expensive operation:

- $\text{card}(R \times S) = \text{card}(R) \times \text{card}(S)$
- $\text{arity}(R \times S) = \text{arity}(R) + \text{arity}(S)$

9/21

Joining relations

Combining Cartesian product and selection

Customer: ID, Name, City, Address

Account: Number, Branch, CustID, Balance

We can join customers with the accounts they own as follows

$$\sigma_{ID=CustID}(Customer \times Account)$$

10/21

Renaming

Gives new names to the attribute of a relation

Syntax: $\rho_{\text{replacement}_1, \dots, \text{replacement}_k}(\text{relation})$

where each **replacement** has the form **old** $\rightarrow$ **new**

Requirements

Let X is the set of attributes of the input relation, then

- The l.h.s. of each replacement belongs to X
- No two replacements have the same l.h.s.
- Composing the identity on X with all replacements results in an injective function

11/21

Examples of renaming

$$\bullet \quad \rho_{B \rightarrow E} \left(\overbrace{\begin{array}{c|ccc} & \mathbf{A} & \mathbf{B} & \mathbf{C} \\ \hline a & b & c \\ 1 & 2 & 3 \end{array}}^R \right) = \begin{array}{c|ccc} & \mathbf{A} & \mathbf{E} & \mathbf{C} \\ \hline a & b & c \\ 1 & 2 & 3 \end{array}$$

- $\rho_{D \rightarrow E}(R)$ and $\rho_{B \rightarrow A}(R)$ are **illegal** (why?)

- $\rho_{A \rightarrow B, B \rightarrow A}(R)$ is **legal** (what does it do?)

- $\rho_{A \rightarrow B, B \rightarrow A, A \rightarrow E}(R)$ is also **illegal** (why?)

12/21

Natural join

Joins two tables for equality on their **common attributes**

Example

Customer: **CustID**, Name, City, Address

Account: Number, Branch, **CustID**, Balance

$\text{Customer} \bowtie \text{Account} \equiv$

$$\pi_{X \cup Y} \left(\sigma_{\text{CustID} = \text{CustID}'} \left(\text{Customer} \times \rho_{\text{CustID} \rightarrow \text{CustID}'}(\text{Account}) \right) \right)$$

where $X = \{ \text{all attributes of Customer} \}$

$Y = \{ \text{all attributes of Account} \}$

14/21

From SQL to relational algebra

SELECT $\mapsto$ projection π

FROM $\mapsto$ Cartesian product $\times$

WHERE $\mapsto$ selection σ

SELECT $A_1, \dots, A_m$
FROM $T_1, \dots, T_n$ $\mapsto \pi_{A_1, \dots, A_m}(\sigma_{\langle \text{condition} \rangle}(T_1 \times \dots \times T_n))$
WHERE $\langle \text{condition} \rangle$

Common attributes in $T_1, \dots, T_n$ must be renamed

15/21

Set operations

Union

R	A	B	$\cup$	S	A	B	=	R $\cup$ S	A	B
	a1	b1			a1	b1			a1	b1
	a2	b2			a3	b3			a2	b2
									a3	b3

Intersection

R	A	B	$\cap$	S	A	B	=	R $\cap$ S	A	B
	a1	b1			a1	b1			a1	b1
	a2	b2			a3	b3				

Difference

R	A	B	$-$	S	A	B	=	R $-$ S	A	B
	a1	b1			a1	b1			a2	b2
	a2	b2			a3	b3				

The relations must have the same set of attributes

16/21

Union and renaming

R	Father	Child	S	Mother	Child
	George	Elizabeth		Elizabeth	Charles
	Philip	Charles		Elizabeth	Andrew
	Charles	William			

We want to find the relation **parent-child**

$\rho_{\text{Father} \rightarrow \text{Parent}}(R) \cup \rho_{\text{Mother} \rightarrow \text{Parent}}(S)$	Parent	Child
	George	Elizabeth
	Philip	Charles
	Charles	William
	Elizabeth	Charles
	Elizabeth	Andrew

17/21

Full relational algebra

Primitive operations: $\pi, \sigma, \times, \rho, \cup, -$

Removing any of these results in a **loss of expressive power**

Derived operations

$\bowtie$ can be expressed in terms of $\pi, \sigma, \times, \rho$

$\cap$ can be expressed in terms difference:

$$R \cap S \equiv R - (R - S)$$

18/21

Other derived operations

Theta-join	$R \bowtie_{\theta} S \equiv \sigma_{\theta}(R \times S)$
Equijoin	$\bowtie_{\theta}$ where θ is a conjunction of equalities
Semijoin	$R \ltimes_{\theta} S \equiv \pi_X(R \bowtie_{\theta} S)$ where X is the set of attributes of R
Antijoin	$R \bar{\ltimes}_{\theta} S \equiv R - (R \ltimes_{\theta} S)$

Why use these operations?

- to write things more succinctly
- they can be optimized independently

19/21

Division

R over set of attributes X

S over set of attributes $Y \subset X$

Let $Z = X - Y$

$$\begin{aligned} R \div S &= \{ r \in \pi_Z(R) \mid \text{for every } s \in S, rs \in R \} \\ &= \{ r \in \pi_Z(R) \mid \{r\} \times S \subseteq R \} \\ &= \pi_Z(R) - \pi_Z(\pi_Z(R) \times S - R) \end{aligned}$$

20/21

Division: Example

Exams		DPT
Student	Course	Course
John	Databases	Databases
John	Networks	Programming
Mary	Programming	
Mary	Math	
Mary	Databases	

$$\text{Exams} \div \text{DPT} = \begin{array}{|c|} \hline \text{Student} \\ \hline \text{Mary} \\ \hline \end{array}$$

$$= \pi_{\text{Student}}(\text{Exams}) - \pi_{\text{Student}}(\pi_{\text{Student}}(\text{Exams}) \times \text{DPT} - \text{Exams})$$